Vulnerability Notes

Access control vulnerabilities

Concepts, Attack Techniques, Detection, and Defenses

Reference
PortSwigger Web Security Academy

Prepared by
Youssef Reda

Contents

1 Introduction to Access Control **1**
1.1 What is Access Control? . . . 1
1.2 Authentication, Session Management, and Access Control . . . 1
1.2.1 Authentication . . . 1
1.2.2 Session Management . . . 1
1.2.3 Authorization and Access Control . . . 1
1.3 The Complete Request Flow . . . 2
1.4 Broken Access Control . . . 2
1.5 Why Access Control Is Difficult . . . 2
1.6 Security Impact . . . 3

2 Access Control Security Models **4**
2.1 Programmatic Access Control . . . 4
2.1.1 Definition . . . 4
2.1.2 How It Works . . . 4
2.1.3 Privilege Matrix . . . 4
2.1.4 Key Characteristics . . . 4
2.1.5 Example . . . 4
2.1.6 Key Point . . . 4
2.2 Discretionary Access Control (DAC) . . . 5
2.2.1 Definition . . . 5
2.2.2 How It Works . . . 5
2.2.3 Example . . . 5
2.2.4 Key Characteristics . . . 5
2.2.5 Advantages and Disadvantages . . . 5
2.2.6 Key Point . . . 5
2.3 Mandatory Access Control (MAC) . . . 5
2.3.1 Definition . . . 5
2.3.2 How It Works . . . 5
2.3.3 Common Example: Military Systems . . . 6
2.3.4 Example . . . 6
2.3.5 Key Characteristics . . . 6
2.3.6 Key Point . . . 6
2.4 Role-Based Access Control (RBAC) . . . 6
2.4.1 Definition . . . 6
2.4.2 How It Works . . . 6
2.4.3 Example . . . 7
2.4.4 Real-World Example . . . 7
2.4.5 Advantages and Disadvantages . . . 7

2.4.6 Key Point . . . 7
2.5 Comparison of Access Control Models . . . 8
2.6 Important Differences . . . 8
2.6.1 DAC vs. MAC . . . 8
2.6.2 Programmatic Access Control vs. RBAC . . . 8
2.7 Quick Memory Reference . . . 8

3 Types of Access Controls 9
3.1 Vertical Access Controls . . . 9
3.1.1 Example . . . 9
3.1.2 Common Examples . . . 9
3.2 Horizontal Access Controls . . . 9
3.2.1 Example . . . 9
3.2.2 Common Examples . . . 10
3.2.3 Relationship with IDOR . . . 10
3.3 Context-Dependent Access Controls . . . 10
3.3.1 Example: Shopping Cart . . . 10
3.3.2 Example: Password Reset . . . 10
3.3.3 Common Contextual Restrictions . . . 11
3.4 Comparison of Access Control Types . . . 11
3.5 Access Control Testing Approach . . . 11
3.5.1 Vertical Access Control . . . 11
3.5.2 Horizontal Access Control . . . 11
3.5.3 Context-Dependent Access Control . . . 12

4 Vertical Privilege Escalation: Unprotected Functionality 13
4.1 Definition . . . 13
4.2 Unprotected Functionality . . . 13
4.3 Discovering Hidden Endpoints . . . 13
4.4 Security by Obscurity . . . 13
4.5 Client-Side Hiding vs. Server-Side Authorization . . . 14
4.6 Testing Workflow . . . 14
4.7 Parameter-Based Access Control . . . 14
4.7.1 Concept . . . 14
4.7.2 Common Examples . . . 14
4.7.3 Root Cause . . . 15
4.7.4 Secure Implementation . . . 15
4.7.5 Testing Workflow . . . 15
4.8 Platform Misconfiguration . . . 15
4.8.1 Concept . . . 15
4.8.2 URL Override Headers . . . 16
4.8.3 HTTP Method-Based Bypass . . . 16

4.8.4 Testing Approach . . . 16
4.8.5 Security Principle . . . 16
4.9 URL-Matching Discrepancies . . . 16
4.9.1 Concept . . . 16
4.9.2 Common Types . . . 17
4.9.3 URL Normalization . . . 17
4.9.4 Testing Methodology . . . 17
4.9.5 Security Principle . . . 18

5 Horizontal Privilege Escalation 19
5.1 Definition . . . 19
5.2 Typical Access Control Failure . . . 19
5.3 Horizontal to Vertical Privilege Escalation . . . 19
5.3.1 Attack Scenario . . . 19
5.3.2 Potential Impact . . . 19
5.3.3 Testing Approach . . . 20
5.4 Insecure Direct Object References (IDOR) . . . 20
5.4.1 Definition . . . 20
5.4.2 Common Object References . . . 20
5.4.3 Example . . . 20
5.4.4 IDOR and Privilege Escalation . . . 20
5.4.5 Predictable and Unpredictable Identifiers . . . 21
5.4.6 Testing Methodology . . . 21
5.5 IDOR Examples . . . 21
5.5.1 Direct Reference to Database Objects . . . 21
5.5.2 IDOR and Privileged Accounts . . . 22
5.5.3 Direct Reference to Static Files . . . 22
5.5.4 Database Objects vs. Static Files . . . 22
5.5.5 Predictable Identifiers . . . 22
5.6 Access Control Vulnerabilities in Multi-Step Processes . . . 23
5.7 Referer-Based Access Control . . . 23
5.8 Location-Based Access Control . . . 23
5.9 How to Prevent Access Control Vulnerabilities . . . 24

Introduction to Access Control

What is Access Control?

Access control is the mechanism that determines whether a user is authorized to perform a specific action on a particular resource. In web applications, access control enforces permissions based on users, roles, resource ownership, and application rules.

Common examples include:

- **Users:** Normal users, managers, and administrators.
- **Resources:** Profiles, orders, reports, and administrative panels.
- **Actions:** Read, create, edit, and delete.

For example, a normal user may be allowed to view and edit their own profile but not access another user's data or delete other users. An administrator may have broader permissions. The purpose of access control is to enforce these restrictions on the server side.

Authentication, Session Management, and Access Control

These three concepts are closely related but serve different purposes.

Authentication

Authentication verifies the identity of a user. For example, when a user submits credentials through a login request, the server validates them to determine which user is attempting to authenticate.

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded
username=ali&password=********
```

Authentication answers:

> Who are you?

Session Management

After authentication, the application needs to associate subsequent HTTP requests with the authenticated user.

For example, the server may issue a session cookie:

```
Set-Cookie: session=abc123
```

The browser includes the cookie in subsequent requests, allowing the server to identify the associated session and user.

Session management answers:

> Which authenticated user does this request belong to?

Authorization and Access Control

Once the application identifies the user, it must determine whether that user is permitted to perform the requested action. For example, an authenticated normal user requesting an administrative resource should be denied access if they lack the required permissions.

```
GET /admin/users HTTP/1.1
Cookie: session=abc123
```

The server must evaluate the user's permissions before granting access.

Authorization answers:

> What is this user allowed to do?

Access control refers to the rules and enforcement mechanisms used to restrict access to resources and functionality according to those permissions.

The Complete Request Flow

A simplified request flow is:

1. **Authentication:** Verify the user's identity.
2. **Session Management:** Associate the request with the authenticated user.
3. **Authorization:** Determine whether the user can perform the requested action on the resource.
4. **Enforcement:** Allow or deny the request based on the authorization decision.

Successful authentication does not imply authorization. A user can be logged in successfully but still be forbidden from accessing administrative functionality or another user's resources.

Broken Access Control

Broken Access Control occurs when an application fails to properly enforce authorization restrictions, allowing users to access resources or perform actions beyond their intended permissions. For example, an application may authenticate a normal user correctly but fail to verify whether that user is authorized to delete another user's account.

```
DELETE /users/50 HTTP/1.1
Cookie: session=abc123
```

If the server processes this request without checking the user's permissions, the application has a Broken Access Control vulnerability. The underlying issue is not necessarily authentication or session management. The application may correctly identify the user while failing to enforce what that user is allowed to do.

Why Access Control Is Difficult

Access control becomes complex as applications introduce different roles, resources, permissions, ownership rules, workflows, and business requirements.

Authorization decisions may depend on several factors:

- The authenticated user's identity and role.
- The specific resource being accessed and its owner.
- The requested action and HTTP method.
- The application's current state and business rules.
- The permissions required by a particular endpoint or API.

Permissions may differ even for resources owned by the same user. For example, a user might be allowed to view an order but not cancel it or request a refund. Access control rules must therefore be enforced consistently across all relevant endpoints and operations. Protecting one interface does not guarantee that equivalent functionality exposed through another endpoint is also protected.

Security Impact

Broken Access Control can allow attackers to perform unauthorized actions, including:

- Accessing sensitive information.
- Viewing or modifying other users' data.
- Accessing administrative functionality.
- Changing application data or business-critical settings.
- Deleting resources or performing other privileged operations.

The severity depends on the permissions bypassed, the resources affected, and the actions the attacker can perform.

Access Control Security Models

Programmatic Access Control

Definition

Programmatic Access Control enforces access control rules through application logic that checks a **privilege matrix** stored in a database or a similar system. The application evaluates the matrix to determine whether a user is authorized to perform a specific action on a resource.

How It Works

1. A user sends an HTTP request.
2. The application identifies the requested resource and action.
3. The application checks the privilege matrix.
4. The request is allowed or denied based on the user's permissions.

Privilege Matrix

A privilege matrix defines which subjects can perform which actions on specific resources. Permissions may be associated with individual users, groups, roles, resources, or workflows.

```
Subject    Resource    Action

Alice      Orders      Read

Alice      Orders      Update

Bob        Orders      Read

Admin      Users       Delete
```

Key Characteristics

- Permission checks are performed by application logic.
- Permissions can be highly granular.
- The model can support complex combinations of users, roles, groups, resources, and workflows.
- Permission data can be stored in a database or a similar system.

Example

Suppose Alice has permission to read Order #100 but does not have permission to delete it.

```
Alice -> Order #100 -> READ   -> Allowed
Alice -> Order #100 -> DELETE -> Denied
```

Key Point

Programmatic Access Control relies on application logic to evaluate a privilege matrix and determine whether an action is permitted.

Discretionary Access Control (DAC)

Definition

Discretionary Access Control (DAC) controls access based on individual users or named groups. Its defining characteristic is the **resource owner's ability to assign or delegate permissions** to other users.

How It Works

The resource owner determines which users can access the resource and what actions they can perform.

1. A resource has an owner.
2. The owner assigns or delegates permissions.
3. Other users receive the specified access rights.

Example

Suppose Youssef owns the file `project.pdf`. He can assign the following permissions:

```
Ahmed -> Read
Ali   -> Edit
Omar  -> No Access
```

Key Characteristics

- Access can be controlled for individual users or groups.
- Resource owners can assign or delegate permissions.
- Permissions can be defined for specific resources and actions.
- Access rights can be highly granular.

Advantages and Disadvantages

Advantage: DAC is flexible because resource owners can determine who has access to their resources. **Disadvantage:** Managing permissions can become complex when many users and resources have different access requirements.

Key Point

DAC allows resource owners to assign or delegate access permissions, subject to any applicable system restrictions.

Mandatory Access Control (MAC)

Definition

Mandatory Access Control (MAC) is a centrally controlled access control model in which access decisions are determined by a central security policy. Unlike DAC, users and resource owners cannot independently modify or delegate permissions in a way that overrides the central policy.

How It Works

1. A central authority defines the security policy.
2. Users and resources are assigned appropriate security classifications or levels.

3. The system evaluates access requests against the policy.
4. Access is allowed or denied according to the policy.

Common Example: Military Systems

Military systems may use security clearance levels such as:

- Public
- Confidential
- Secret
- Top Secret

Documents may also have corresponding security classifications.

```
Document A -> Confidential
Document B -> Secret
Document C -> Top Secret
```

A user can access a document only when the centralized security policy permits it.

Example

Ahmed has Confidential clearance but attempts to access a Secret document.

```
Ahmed -> Confidential Clearance
Ahmed -> Access Secret Document
      -> Denied
```

Even if Ahmed owns the document, he cannot override the central security policy or grant himself additional permissions.

Key Characteristics

- Access decisions are controlled by a central security policy.
- Users cannot independently modify their access rights.
- Resource owners cannot delegate permissions in violation of the policy.
- MAC is commonly associated with military and other high-security environments.

Key Point

MAC enforces centrally defined security restrictions that users and resource owners cannot override.

Role-Based Access Control (RBAC)

Definition

Role-Based Access Control (RBAC) assigns permissions to roles, and users are assigned to one or more roles. Instead of assigning permissions individually to every user, RBAC associates users with roles that already have defined permissions.

$$\text{User} \rightarrow \text{Role} \rightarrow \text{Permissions}$$

How It Works

1. Define roles according to organizational responsibilities.

2. Assign permissions to each role.
3. Assign users to the appropriate roles.
4. The application determines the user's effective permissions through their assigned roles.

Example

Consider an application with the following roles and permissions:

- **Admin:** Create users, delete users, edit users, and view reports.
- **Manager:** Approve requests and view reports.
- **Employee:** View their own profile and create requests.
- **Auditor:** Permissions defined according to auditing responsibilities.

Users are assigned to these roles as follows:

```
Youssef -> Employee
Ahmed   -> Manager
Ali     -> Admin
```

Real-World Example

A company may define a `Purchase Clerk` role with permissions to:

- View the purchase ledger.
- Create purchase orders.
- Update purchase orders.

Multiple employees can be assigned to this role and receive its permissions. When an employee joins the company, assigning the role grants the corresponding permissions. When the employee leaves, removing the role revokes the permissions obtained through that role, assuming no other assignments grant the same access.

Advantages and Disadvantages

Advantages:

- Easier to manage than assigning permissions to every user individually.
- Suitable for complex applications and organizations.
- Simplifies onboarding and offboarding.
- Provides effective permission management when roles are designed appropriately.

Disadvantage: Creating too many roles can increase complexity and make the system difficult to manage. The objective is to balance sufficient permission granularity with a manageable number of roles.

Key Point

RBAC assigns permissions to roles and assigns users to those roles, simplifying the administration of access rights.

Comparison of Access Control Models

Model	Main Idea	Access Control
Programmatic	The application checks a privilege matrix.	Application logic
DAC	Resource owners assign or delegate permissions.	Resource owner
MAC	A central security policy determines access.	Central authority and policy
RBAC	Permissions are assigned to roles.	Role and policy administrators

Important Differences

DAC vs. MAC

- **DAC:** Resource owners can assign or delegate permissions within applicable restrictions.
- **MAC:** Resource owners cannot override the centrally enforced security policy.

Programmatic Access Control vs. RBAC

- **Programmatic Access Control:** Describes enforcing permissions through application logic that checks a privilege matrix.
- **RBAC:** Defines how permissions are organized and assigned through roles.

These concepts are not necessarily mutually exclusive. An application can implement RBAC programmatically by checking a user's roles and their associated permissions.

Quick Memory Reference

- **Programmatic:** Application logic checks permissions.
- **DAC:** Resource owner assigns permissions.
- **MAC:** Central security policy controls access.
- **RBAC:** Users receive permissions through roles.

Types of Access Controls

Vertical Access Controls

Vertical access controls restrict access to functionality based on a user's privilege level or role. Different users may be permitted to perform different actions depending on their assigned permissions. For example, a normal user may view and edit their own profile, while an administrator may also manage users, modify security settings, and access administrative functionality. A vertical access control vulnerability occurs when a user can access functionality intended for a higher-privileged role.

Example

Consider an administrative endpoint:

```
GET /admin/users HTTP/1.1
```

A normal user should be denied access if they lack the required permissions. If the server instead returns the administrative functionality, the user has bypassed vertical access controls.

Common Examples

- Accessing an administrative panel as a normal user.
- Invoking administrative API endpoints without sufficient privileges.
- Deleting users through functionality restricted to administrators.
- Modifying security settings without the required permissions.
- Accessing functionality intended only for managers or staff.

Key concept: Vertical access control concerns differences in privilege levels. Moving from a lower privilege level to a higher one is known as **vertical privilege escalation**.

Horizontal Access Controls

Horizontal access controls restrict access to resources based on resource ownership or the specific users authorized to access them. Users may have the same privilege level and access the same types of functionality, but each user should only be able to access resources they are authorized to use.

Example

Suppose two users have separate accounts:

- Ahmed owns account `100` and its associated transactions.
- Mohamed owns account `200` and its associated transactions.

Ahmed should be able to access his own account but not Mohamed's account.

The application uses the following request:

```
GET /account?id=100 HTTP/1.1
```

If Ahmed changes the identifier:

```
GET /account?id=200 HTTP/1.1
```

and receives Mohamed's account information without proper authorization, the application has a horizontal access control vulnerability.

Common Examples

- Viewing another user's private profile.
- Accessing another user's orders or transactions.
- Reading private messages belonging to another user.
- Downloading another user's files.
- Modifying another user's account information.

Relationship with IDOR

Horizontal access control vulnerabilities are commonly associated with **Insecure Direct Object References (IDOR)**.

For example:

```
GET /api/users/123/profile HTTP/1.1
```

If changing the user identifier from `123` to `124` allows unauthorized access to the other user's private information, this may indicate an IDOR vulnerability. IDOR is a common mechanism behind horizontal access control failures, although not every horizontal access control vulnerability is necessarily an IDOR. **Key concept:** Horizontal access control concerns access to resources belonging to different users at the same privilege level. The attacker does not necessarily gain a higher role.

Context-Dependent Access Controls

Context-dependent access controls restrict actions based on the application's current state, workflow, or the completion of required steps. A user may generally be permitted to perform an action, but the application must ensure that the action is allowed at the current stage of the workflow.

Example: Shopping Cart

Consider a checkout workflow:

1. Add items to the cart.
2. Review the cart.
3. Complete payment.
4. Finalize the order.

Modifying the cart may be permitted before payment, but the application should enforce the appropriate restrictions once the transaction has been completed. If a user can bypass these restrictions and modify a completed order, the application may have a context-dependent access control vulnerability.

Example: Password Reset

A secure password-reset workflow may require the following steps:

1. Request a password reset.
2. Receive a one-time password (OTP).
3. Verify the OTP.

4. Change the password.

The application must not allow the password to be changed before the required verification has been completed. If an attacker skips the verification step and directly accesses the password-change functionality, this may constitute a context-dependent access control bypass.

Common Contextual Restrictions

Access decisions may depend on:

- Previous user actions.
- Current workflow or transaction state.
- Payment or order completion status.
- Identity or security verification status.
- Required approvals.
- Whether a process has already been completed.

Key concept: Context-dependent access control determines whether an action is permitted in the application's current state, rather than relying solely on the user's role or resource ownership.

Comparison of Access Control Types

Type	Main Question	Example
Vertical	Does the user have sufficient privileges?	A normal user accesses an admin endpoint.
Horizontal	Is the user authorized to access this specific resource?	Ahmed accesses Mohamed's account.
Context-dependent	Is the action permitted in the current application state?	A user modifies an order after payment.

Table 1: Comparison of access control types

Access Control Testing Approach

When testing for Broken Access Control, evaluate the application from three perspectives.

Vertical Access Control

Determine whether a low-privileged user can access functionality restricted to a higher-privileged user.

```
Normal User -> Restricted Admin Endpoint
```

Horizontal Access Control

Determine whether one user can access or modify resources belonging to another user without authorization.

```
User A -> User B's Resource
```

Context-Dependent Access Control

Determine whether a user can skip required workflow steps or perform an action in an invalid application state.

```
Skip Required Step
       |
       v
Perform Restricted Action
```

Testing should verify the actual authorization failure and its impact, rather than treating a changed URL, identifier, or workflow step alone as proof of a vulnerability.

Vertical Privilege Escalation: Unprotected Functionality

Definition

Vertical Privilege Escalation occurs when a low-privileged user gains access to functionality or resources intended for users with higher privileges. A common cause is missing or improperly enforced server-side authorization.

Unprotected Functionality

An application may expose administrative endpoints without verifying the requesting user's privileges. For example, an application may hide the admin panel link from normal users while leaving the endpoint accessible:

```
GET /admin HTTP/1.1
Host: insecure-website.com
```

If the server does not enforce authorization, a normal user may access the administrative functionality directly. A properly protected endpoint must verify the user's privileges and deny unauthorized requests, typically with `403 Forbidden`.

Discovering Hidden Endpoints

Administrative functionality may be exposed through predictable paths such as:

```
/admin
/administrator
/admin-panel
/manage
/dashboard
```

Testers can use content discovery and wordlists to identify endpoints that are not linked from the user interface.

Other discovery sources include:

- `/robots.txt`, which may disclose paths through entries such as `Disallow: /admin`.
- HTML source code and client-side JavaScript.
- Application responses and leaked links.

The `robots.txt` file provides instructions to compliant crawlers; it does not restrict user access. **Important distinction:** Content discovery identifies whether an endpoint exists, while authorization testing determines whether the current user is permitted to access it. A hidden endpoint is not itself a vulnerability.

Security by Obscurity

Using an unpredictable URL, such as `/administrator-panel-yb556`, is not a substitute for access control. The path may still be discovered through JavaScript, HTML, application responses, or content discovery.

$$\text{Hidden URL} \neq \text{Authorization}$$

The server must enforce permissions regardless of whether the endpoint's URL is known or difficult to guess.

Client-Side Hiding vs. Server-Side Authorization

Client-side code may conditionally display an administrative link:

```
if (isAdmin) {
    // Display the admin panel link
}
```

However, hiding the link only changes the interface. The URL may remain visible in the delivered JavaScript, and users can send HTTP requests directly without using the interface. Therefore, authorization must be enforced server-side for every protected request.

Testing Workflow

1. Log in as a low-privileged user.
2. Identify potentially sensitive endpoints through content discovery and client-side inspection.
3. Request the endpoints directly.
4. Examine the responses and attempt to access the protected functionality.
5. Confirm whether the server correctly denies unauthorized access.

A successful response alone does not necessarily prove a vulnerability. Confirm that the low-privileged user can access or perform an action that should be restricted to higher-privileged users.

Parameter-Based Access Control

Concept

Parameter-based access control is vulnerable when an application makes authorization decisions using client-controlled values, allowing users to manipulate their privileges.

Such values may appear in:

- Hidden HTML fields.
- Cookies.
- Query string parameters.
- Request body parameters or HTTP headers.

Common Examples

Hidden fields:

```
<input type="hidden" name="admin" value="false">
```

A user can modify the field before submitting the request:

```
<input type="hidden" name="admin" value="true">
```

Cookies:

```
Cookie: admin=false
```

A vulnerable application might accept a modified value:

```
Cookie: admin=true
```

Query parameters:

```
GET /home?role=0 HTTP/1.1
```

If the application maps `role=0` to a normal user and `role=1` to an administrator, modifying the parameter may bypass authorization.

Root Cause

The vulnerability occurs when the server trusts user-controlled input to determine authorization.

$$\text{Client-Controlled Value} \rightarrow \text{Authorization Decision}$$

Hidden fields and cookies are not inherently insecure; the problem is relying on modifiable client-side values as authoritative proof of privileges.

Secure Implementation

The server should identify the authenticated user through a trusted session and determine their permissions using trusted server-side data.

1. Authenticate the user.
2. Identify the account associated with the session.
3. Retrieve the user's actual roles and permissions.
4. Check authorization for the requested action and resource.
5. Allow or deny the request accordingly.

Client-provided parameters may specify the requested action, but they must not independently grant privileges.

Testing Workflow

1. Log in as a low-privileged user.
2. Inspect requests for privilege-related values, such as `admin`, `isAdmin`, `role`, or `privilege`.
3. Test whether these values can be modified in cookies, parameters, hidden fields, or request bodies.
4. Observe whether the changes grant access to restricted functionality.

If manipulating a parameter allows a low-privileged user to perform an action reserved for administrators, the application may be vulnerable to vertical privilege escalation.

Platform Misconfiguration

Concept

Some applications enforce access control at the platform layer, such as a front-end server or reverse proxy, using rules based on URL paths, HTTP methods, and user groups. For example, a platform rule may deny managers access to a specific administrative endpoint. If the platform and the application interpret the same request differently, these restrictions may be bypassed, resulting in **vertical privilege escalation**. Two common techniques are URL override headers and HTTP method manipulation.

URL Override Headers

Some frameworks support headers that influence how the application interprets the requested URL, including:

```
X-Original-URL: /admin/deleteUser
X-Rewrite-URL: /admin/deleteUser
```

Suppose the platform blocks requests to `/admin/deleteUser`. A tester can check whether the following request is interpreted differently by the platform and the application:

```
POST / HTTP/1.1
Host: example.com
X-Original-URL: /admin/deleteUser
```

If the platform checks the original path (`/`) while the application processes the overridden path (`/admin/deleteUser`), the request may bypass the intended restriction. The vulnerability depends on an actual authorization bypass; support for these headers alone is not sufficient to establish one.

HTTP Method-Based Bypass

Access control rules may restrict a sensitive action for a particular HTTP method without protecting equivalent requests using other supported methods. For example, a platform might block:

```
POST /admin/deleteUser?id=123 HTTP/1.1
```

A tester can check whether the application also performs the same action through another method:

```
GET /admin/deleteUser?id=123 HTTP/1.1
```

If the alternative request performs the restricted action without proper authorization, the access control can be bypassed.

Testing Approach

When testing restricted functionality:

1. Identify an endpoint that is correctly blocked for a low-privileged user.
2. Test URL override headers such as `X-Original-URL` and `X-Rewrite-URL`.
3. Test alternative HTTP methods supported by the application.
4. Compare responses and verify whether the sensitive action actually succeeds.

Security Principle

Authorization must be enforced consistently across the application stack. The application must validate the user's permissions for the actual resource and action being requested, regardless of URL representation or HTTP method.

URL-Matching Discrepancies

Concept

URL-matching discrepancies occur when the access control layer and the application router interpret the same URL differently. The access control layer may allow a request that the application maps to restricted functionality, resulting in an access control bypass and potentially vertical privilege

escalation.

Common Types

1. Case sensitivity An application may treat paths as case-insensitive, while the access control mechanism uses case-sensitive matching.

```
GET /ADMIN/DELETEUSER HTTP/1.1
```

The application might map this path to `/admin/deleteUser`, while the access control layer fails to recognize it as a protected endpoint. **2. Suffix pattern matching** Some frameworks may map paths containing file extensions to the same endpoint as the original path.

For example, in configurations that support suffix pattern matching:

```
/admin/deleteUser.anything
```

may be routed to:

```
/admin/deleteUser
```

In Spring MVC, suffix pattern matching was enabled by default in versions before Spring 5.3. Actual behavior depends on the framework version and configuration. **3. Trailing slash**

The access control layer and application router may treat these paths differently:

```
/admin/deleteUser
/admin/deleteUser/
```

If the application maps both paths to the same restricted functionality but the access control layer protects only one, the alternative path may bypass the restriction.

URL Normalization

Web servers, access control mechanisms, and application routers may normalize URLs differently. Relevant differences include:

- Letter case.
- File extensions or suffixes.
- Trailing slashes.
- URL encoding and path normalization.

The underlying weakness is inconsistent interpretation:

$$\text{Authorization Matching} \neq \text{Application Routing}$$

Testing Methodology

For a protected endpoint, test alternative URL representations, such as:

```
/admin/deleteUser
/ADMIN/DELETEUSER
/admin/deleteUser.anything
/admin/deleteUser/
```

Determine whether:

1. The access control layer allows the alternative request.
2. The application routes it to the protected endpoint.
3. The restricted functionality can actually be accessed or executed.

A routing difference alone is not a vulnerability. The issue is confirmed when inconsistent URL matching allows an unauthorized user to access or perform a restricted action.

Security Principle

The access control mechanism must apply authorization consistently with the application's URL parsing, normalization, and routing behavior. Every representation that resolves to a protected resource must receive the appropriate authorization checks.

Horizontal Privilege Escalation

Definition

Horizontal privilege escalation occurs when a user accesses resources belonging to another user at the same privilege level. Unlike vertical privilege escalation, the attacker does not gain a higher role but crosses user-level access boundaries.

- **Vertical:** Normal user accesses administrator functionality.
- **Horizontal:** User A accesses User B's resources.

For example, if Ahmed can access Mohamed's account data while authenticated as Ahmed, the application has a horizontal access control vulnerability.

Typical Access Control Failure

Consider an application that identifies accounts using a user-controlled parameter:

```
GET /myaccount?id=123 HTTP/1.1
Host: insecure-website.com
```

Suppose `id=123` belongs to Ahmed and `id=124` belongs to Mohamed. If Ahmed changes the parameter:

```
GET /myaccount?id=124 HTTP/1.1
Host: insecure-website.com
```

the application must verify that Ahmed is authorized to access the requested account. If it returns Mohamed's private data without authorization, the application is vulnerable. The core issue is a missing or incorrect authorization check between the authenticated user and the requested resource.

Horizontal to Vertical Privilege Escalation

A horizontal access control vulnerability can lead to vertical privilege escalation if the attacker gains access to an account with higher privileges.

Attack Scenario

Consider an application that retrieves accounts using a user-controlled parameter:

```
GET /myaccount?id=123 HTTP/1.1
```

If the attacker can modify the identifier to access another user's account, the impact depends partly on the privileges of the target account.

```
/myaccount?id=123  -> Attacker's account
/myaccount?id=456  -> Another user's account
```

If account `456` belongs to an administrator, the horizontal access control flaw may expose privileged resources or enable further compromise of the administrator's account.

Potential Impact

Depending on the application's implementation, accessing a privileged user's account may expose administrative data, credentials, or functionality. However, accessing another user's data does not

automatically grant the attacker's session the target user's privileges; additional conditions may be required.

Testing Approach

When identifying a horizontal access control vulnerability, determine the target account's role and assess the impact of the unauthorized access. Potentially privileged targets include administrators, managers, moderators, and staff members.

Horizontal Access Control Bypass → Privileged Account Exposure → Potential Vertical Escalation

Horizontal privilege escalation describes unauthorized access to another user's resources, while vertical privilege escalation involves gaining access to functionality or permissions above the attacker's own privilege level.

Insecure Direct Object References (IDOR)

Definition

Insecure Direct Object Reference (IDOR) is an access control vulnerability that occurs when an application uses a user-controlled reference to access an object without properly verifying whether the current user is authorized to access it. **Core pattern:**

$$\text{Object Reference} + \text{Missing Authorization} \rightarrow \text{Unauthorized Access}$$

Common Object References

An object is any resource managed by an application, such as a user account, order, invoice, message, document, image, file, or profile.

Applications may identify these objects using:

```
?id=123
?user_id=456
?order_id=789
?file_id=100
```

Other references include UUIDs, GUIDs, filenames, and identifiers embedded in URL paths or API requests.

Example

A user accesses their own account:

```
GET /myaccount?id=123 HTTP/1.1
```

The user modifies the identifier:

```
GET /myaccount?id=124 HTTP/1.1
```

If the server returns account `124` without checking whether the authenticated user has permission to access it, the application is vulnerable to IDOR. The secure implementation must authorize access to the requested object before returning its data or performing the requested action.

IDOR and Privilege Escalation

IDOR is a vulnerability mechanism, whereas privilege escalation describes a possible impact.

- **Horizontal privilege escalation:** Accessing another user's resources at the same privilege level.
- **Vertical privilege escalation:** Gaining access to functionality or resources restricted to users with higher privileges.

An IDOR may lead to either outcome, depending on the resource exposed and the permissions involved. Accessing an administrator's resource does not automatically grant administrative privileges.

Predictable and Unpredictable Identifiers

Sequential identifiers can make it easier to discover references to other resources:

```
/account?id=123
/account?id=124
/account?id=125
```

Applications may instead use UUIDs or GUIDs, making identifiers harder to guess. However, an unpredictable identifier does not prevent IDOR if an attacker obtains a valid reference and the server fails to enforce authorization.

$$\text{Unpredictable Identifier} \neq \text{Authorization}$$

The security boundary must be enforced by server-side authorization checks, not by the secrecy or complexity of object identifiers.

Testing Methodology

When testing for IDOR:

1. Identify requests that reference specific objects.
2. Modify the object reference or use another valid identifier discovered within the application.
3. Observe whether the server accepts the modified request.
4. Verify whether another user's protected data is exposed or an unauthorized action is performed.
5. Confirm that the application fails to enforce the required authorization policy.

Common parameters and references to investigate include:

```
id
user_id
account_id
order_id
message_id
document_id
file_id
```

Also inspect identifiers in URL paths, API requests, and other application features. **Important:** A user-controlled identifier alone is not a vulnerability. IDOR exists when insufficient authorization allows access to an object or operation the user is not permitted to access.

IDOR Examples

Direct Reference to Database Objects

An IDOR can occur when an application uses a user-controlled parameter to retrieve a database record without verifying whether the authenticated user is authorized to access it.

For example:

```
GET /customer_account?customer_number=132355 HTTP/1.1
```

The application may use the parameter to retrieve a customer record. If changing the reference returns another customer's data without authorization, the application is vulnerable.

```
GET /customer_account?customer_number=132356 HTTP/1.1
```

This may result in horizontal privilege escalation by allowing one customer to access another customer's information.

IDOR and Privileged Accounts

An IDOR may expose resources belonging to privileged users. Depending on the resource and the application's functionality, this could disclose sensitive information or expose privileged operations. When assessing impact, determine who owns the targeted resource and what permissions or sensitive information are associated with it. Accessing a privileged user's resource does not, by itself, grant the attacker that user's privileges.

Direct Reference to Static Files

IDOR can also affect files stored on the server, such as chat transcripts or documents, when the application exposes them through predictable or user-controlled paths.

For example:

```
GET /static/12144.txt HTTP/1.1
```

If changing the filename accesses another user's transcript:

```
GET /static/12145.txt HTTP/1.1
```

and the server returns the file without enforcing the required access restrictions, this is an IDOR. Such files may contain private messages, personal information, credentials, or internal data, potentially leading to further compromise.

Database Objects vs. Static Files

- **Database object:** A parameter such as `customer_number` identifies a database record.
- **Static file:** A filename or URL path identifies a stored file.

Both can be vulnerable when access to the referenced resource is not properly authorized. Files served directly by a web server may require separate access controls because application-level checks can be bypassed if the files are publicly accessible.

Predictable Identifiers

Sequential identifiers and filenames can make unauthorized references easier to discover. However, predictability is not the root cause of IDOR.

$$\text{User-Controlled Reference} + \text{Insufficient Authorization} \rightarrow \text{Unauthorized Access}$$

Unpredictable identifiers do not eliminate the vulnerability if an attacker obtains a valid reference and the server fails to enforce authorization.

Access Control Vulnerabilities in Multi-Step Processes

Multi-step processes may enforce access controls on some steps but omit them from others. An attacker can bypass the protected steps by directly submitting the request for an unprotected step with the required parameters.

Testing Workflow:

1. Capture the requests involved in the multi-step process using Burp Suite.
2. Identify the request responsible for the final action, such as confirming changes.
3. Replay this request using a low-privileged user's session and the required parameters.
4. Verify whether the protected action succeeds without proper authorization.

Key Point: Every request performing a privileged action must enforce authorization independently, regardless of whether previous workflow steps were completed.

Referer-Based Access Control

Some applications rely on the HTTP `Referer` header to control access to sensitive endpoints instead of verifying the user's permissions. An attacker can modify this header to impersonate a request originating from an authorized page and bypass access controls.

Testing Workflow:

1. Log in using a low-privileged account and capture a request to a sensitive endpoint.
2. Send the request to Burp Repeater.
3. Modify the `Referer` header to point to the expected authorized page, such as `/admin`.
4. Resend the request and verify whether the action succeeds without proper authorization.

Example:

```
POST /admin/deleteUser HTTP/1.1
Host: target.com
Referer: https://target.com/admin
Cookie: session=USER_SESSION
Content-Type: application/x-www-form-urlencoded
username=carlos
```

Key Point: The `Referer` header is client-controlled and must never be used as a substitute for server-side authorization checks.

Location-Based Access Control

Some applications restrict access to resources based on the user's geographical location, for example, due to legal requirements or regional content restrictions. These controls may be bypassed when applications rely on weak or client-controlled location information.

Testing Workflow:

1. Identify how the application determines the user's location, such as IP geolocation or browser-provided geolocation data.
2. Test access from permitted and restricted locations using an authorized testing environment.
3. Inspect requests and responses in Burp Suite to determine whether geographical restrictions are enforced server-side.

4. Test whether manipulating client-side location data or changing the apparent IP location bypasses the restriction.

Key Point: Applications must enforce geographical restrictions server-side using an appropriate, trustworthy source of location information. A VPN or proxy bypass alone does not necessarily indicate a vulnerability.

How to Prevent Access Control Vulnerabilities

Access control vulnerabilities can be prevented by applying a defense-in-depth approach and enforcing authorization consistently across the application.

1. **Never Rely on Obfuscation:** Hiding URLs or using unpredictable paths does not prevent unauthorized access. Always enforce authorization on the server.
2. **Deny Access by Default:** Deny access to resources unless explicitly permitted. Public access should only be allowed when intended.
3. **Use a Centralized Access Control Mechanism:** Implement a consistent, application-wide mechanism, such as middleware or authorization policies, to reduce inconsistent or missing checks.
4. **Enforce Explicit Access Rules:** Require developers to define authorization rules for protected resources and actions. Authentication alone does not guarantee permission to perform an action.
5. **Audit and Test Access Controls:** Regularly test authorization rules for horizontal and vertical privilege escalation, direct endpoint access, multi-step process bypasses, and HTTP method or header manipulation.

Key Point: Authentication identifies the user; authorization determines what the user is allowed to do. Every protected action must enforce appropriate server-side authorization.